

AI224: Operating Systems — Lecture 10

Dr. Karim Lounis

Jan - May 2025



Memory Management

Memory Management (Con't)

Page size trade off

To reduce the amount of internal fragmentation per page, it is often suggested to use **small page sizes**.

However, by reducing the size of the pages, there will be an overhead in the size of the page tables.

Reducing page table sizes implies using **large page sizes**.

Nowadays, page size tend to be bigger as the main memory and processes are getting bigger.

Today's page sizes are typically 4KB and 8KB, other systems use bigger page sizes e.g., Solaris uses page sizes of 8KB and 4MB

Process execution and paging

When a process is ready to be executed, its size, expressed in pages, is examined.

i.e., its virtual address space is divided into a certain number of pages.

Each page of the process needs one frame in the physical memory.

i.e., Each page has its dedicated frame in the memory.

If a processes requires **N** pages to be loaded, then at least **N** frames should be available in the main memory.

For each page that is loaded into an allocated frame, the frame's base address is put into the page table of that process.

The OS is aware of the memory status, i.e., which frames are available and which frames are allocated. It uses a system data structure called:
frame table.

Address Translation Hardware

The hardware that manipulates page tables for address translation can be implemented in several ways:

Usually, when the page tables are small (e.g., 256 entries), the hardware is implemented as a set of registers PTR (Page Table Registers).

However, when page tables are very large (1M entries) such as the ones used in contemporary computers, the use of registers **is not feasible**.

Rather, page tables are stored in the main memory, and are pointed out by a dedicated register PTBR (Page Table Base Register).

PTBR (Page Table Base Register) is kept in the process PCB and is loaded into the CPU at context-switch.

The problem with this last scheme is the **time required** to access a user memory location: **As for each access to location ℓ , we need to access the page table first to locate the corresponding frame, and then access the location ℓ in RAM.**

TLB (Translation Look-aside Buffer)

The standard solution to the previous problem, is to use a special, small, fast-lookup hardware cache called TLB (Translation Look-aside Buffer).

Each entry in the TLB is composed of two parts: a key (page number) and a value (frame number).

A TLB is typically small, having between 32 and 1024 entries.

In certain CPUs, they implement two types of TLBs, one for instructions (TLBi) and another for data (TLBd).

The TLB stores the most recent pages table entries (the ones that have been recently requested).

The TLB must be flushed (or erased) when a context-switch occurs.

TLB (Translation Look-aside Buffer)

The standard solution to the previous problem, is to use a special, small, fast-lookup hardware cache called TLB (Translation Look-aside Buffer).

Each entry in the TLB is composed of two parts: a key (page number) and a value (frame number).

A TLB is typically small, having between 32 and 1024 entries.

In certain CPUs, they implement two types of TLBs, one for instructions (TLBi) and another for data (TLBd).

The TLB stores the most recent pages table entries (the ones that have been recently requested).

The TLB must be flushed (or erased) when a context switch occurs.

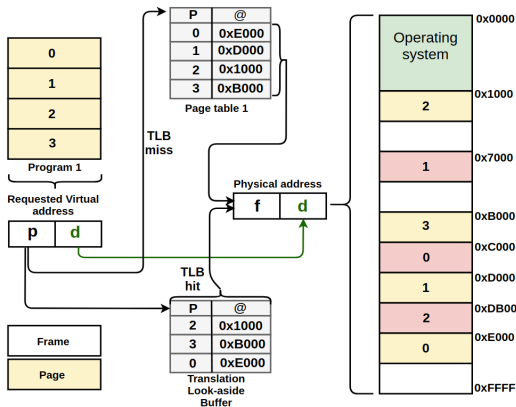
Otherwise, there will be valid logical addresses pointing to wrong or invalid physical addresses.

Paging: Paging with TLB miss

Paging with TLB miss

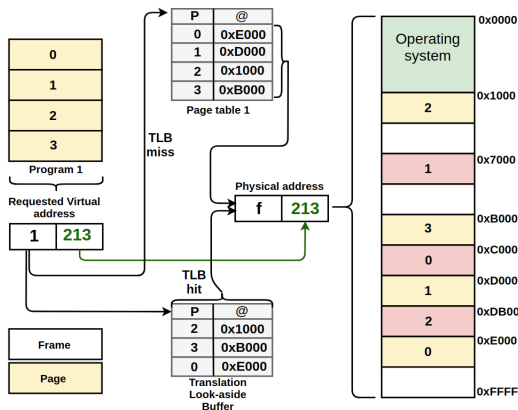
TLB (Translation Look-aside Buffer)

The standard solution to the previous problem, is to use a special, small, fast-lookup hardware cache called TLB (Translation Look-aside Buffer).



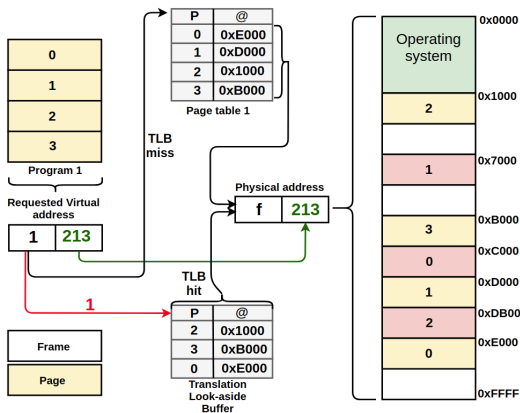
TLB (Translation Look-aside Buffer)

Let us assume that program 1 is executing and the CPU requested access to the memory location pointed out by the virtual address [0x1213]:



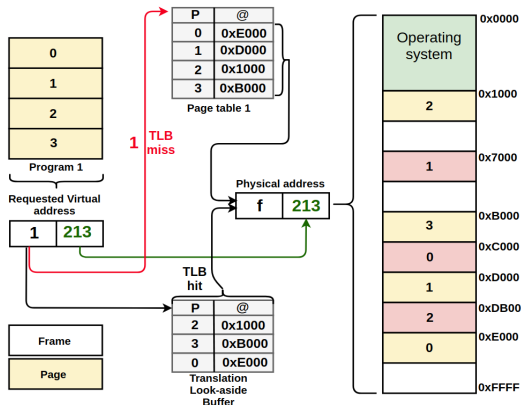
TLB (Translation Look-aside Buffer)

The page number of the the virtual address [0x1213], in the case 1, is first presented to the TLB:



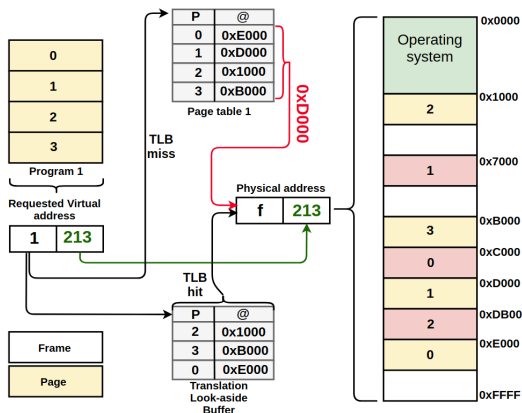
TLB (Translation Look-aside Buffer)

The page number is not in the TLB (a.k.a., TLB miss), the CPU access the page table in the memory to look for the corresponding frame of that page number 1:



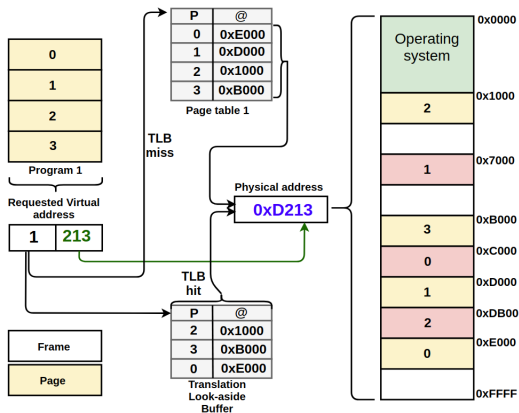
TLB (Translation Look-aside Buffer)

The page number is found in the page table, and the corresponding frame base address is returned:



TLB (Translation Look-aside Buffer)

Using the returned frame base address and the offset, a physical address is generated:

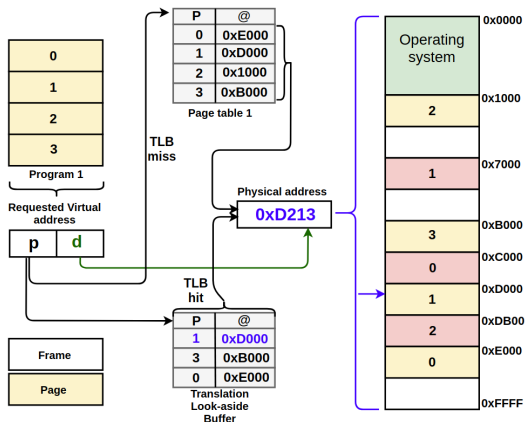


The physical address is then placed in the MAR (Memory Address Register) to read/write into that physical memory location:



TLB (Translation Look-aside Buffer)

The missed page number and its corresponding frame base address are added into the TLB for a quick fetch on the next reference:

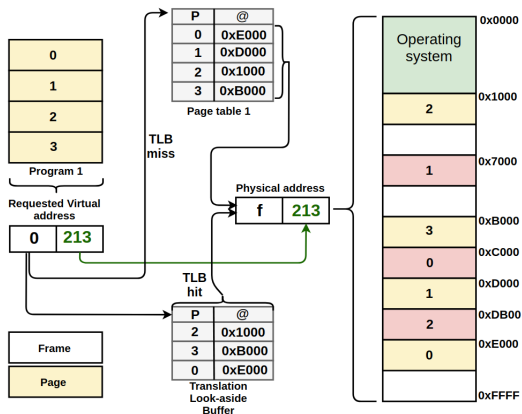


Paging: Paging with TLB hit

Paging with TLB hit

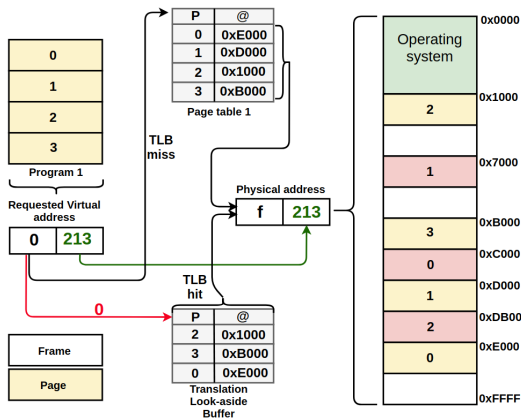
TLB (Translation Look-aside Buffer)

Let us assume that program 1 is executing and the CPU requested access to the memory location pointed out by the virtual address [0x0213]:



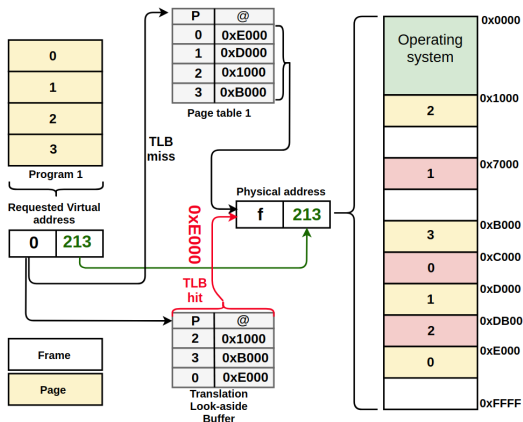
TLB (Translation Look-aside Buffer)

The page number of the the virtual address [0x0213], in the case 0, is first presented to the TLB:



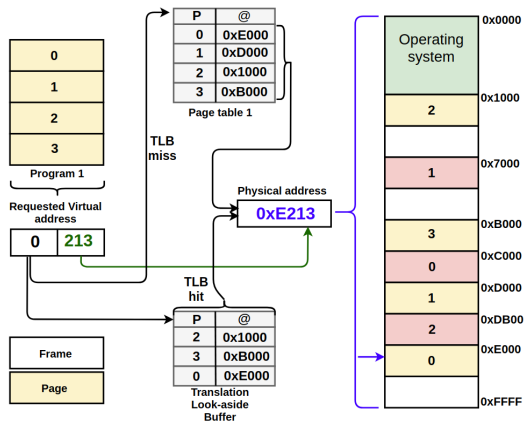
TLB (Translation Look-aside Buffer)

The page number 0 is in the TLB (a.k.a., TLB hit), the frame base address is returned for physical address construction:



TLB (Translation Look-aside Buffer)

The physical address is then placed in the MAR (Memory Address Register) to read/write into that physical memory location:



Effective Memory Access Time

In a system that uses paging along with TLB, the effective memory access time is given as follow:

$$t_{effective} \approx (P_{TLB-hit} \times t_{access}) + ((1 - P_{TLB-hit}) \times (2 \times t_{access}))$$

Where:

- $P_{TLB-hit}$ is the hit ratio (or proba of finding a page number in the TLB).
- t_{access} is the access time for one memory access.

Example.

If the hit ratio is 80% and the access time of one memory access is 100 *ns*, then the effective memory access time is:

$$t_{effective} \approx (0.8 \times 100) + (0.2 \times 200) \approx 120 \text{ ns}$$

Virtual address space and Memory address space

Operating systems are designed and implemented to run on a particular hardware architecture.

i.e., a 32-bit operating system (e.g., Windows 95, 98, XP) can only run on computers that have a 32-bit CPU or greater (64-bit CPU).

In a **N**-bit operating system, each user program sees an **N**-bit virtual address space (i.e., 2^N virtual addresses).

e.g., on a 32-bit operating system, programs have their own $2^{32}=4\text{GB}$ of virtual address space.

Yet, a computer may have less available memory, e.g., 256MB of RAM.

Virtual address space and Memory address space

In a computer that has 256MB of RAM and uses 4KB of page size:

The RAM is divided into $256\text{MB}/4\text{KB} = 64\text{K}$ frames.

This means that we will need **16 bits** to address 64K frames.

The memory is 256MB, which implies physical addresses on 28 bits.

i.e., physical address $\equiv (b_0, b_1, \dots, b_{27})_2$.

By dividing 256MB into 64K frames of 4KB, we need **12 bits** for offset.

i.e., physical address $\equiv (b_0, b_1, \dots, b_{15}, b_{16}, b_{17}, \dots, b_{27})_2$.

The 16 most significant bits $(b_0, b_1, \dots, b_{15})_2$ will be used to locate a frame in the memory, whereas the lower 12 bits $(b_{16}, b_{17}, \dots, b_{27})_2$ will be used as an offset within a frame.

Virtual address space and Memory address space

Thus, we have each physical memory address is in the form of:

$$\underbrace{(b_0, b_1, \dots, b_{15})}_{\text{frame number}}, \underbrace{(b_{16}, b_{17}, \dots, b_{27})}_{\text{displacement}})_2$$

Now, considering a 32-bit operating system, each user program will have a 4GB of virtual memory space and may generate virtual addresses on 32 bits.

$$\text{i.e., virtual address} \equiv (b_0, b_1, \dots, b_{31})_2.$$

And since each program is divided into pages of 4KB, we will have 4GB/4KB = 1M pages per program. We need **20 bits** to locate pages.

$$\underbrace{(b_0, b_1, \dots, b_{19})}_{\text{page number}}, \underbrace{(b_{20}, b_{21}, \dots, b_{31})}_{\text{displacement}})_2$$

So the virtual address space **may be larger/smaller than** the physical address space **i.e., we can have more/less pages than frames (or equal)**.

This means, physical address and virtual addresses may not always have the same size. How does the translation (mapping) occur then?

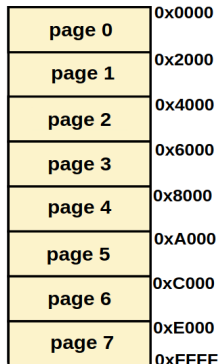
Virtual address space and Memory address space

Example. Consider a 16-bit operating system. Each program sees a 2^{16} addresses. If we consider a page size of **8KB**, the virtual address space will be divided into **8 pages**:

Program view of the
virtual address space



OS view of the virtual
address space

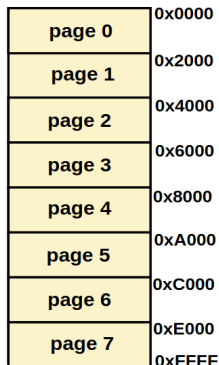


Virtual address space and Memory address space

Program view of the virtual address space



OS view of the virtual address space

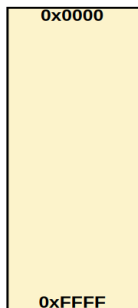


So we use **3 bits** for page numbers and **13 bits** for offset.

Virtual address space and Memory address space

Given a virtual address **[0x325A]** that translate into the physical address **[0x0735A]**. We examine the virtual address such that: the first **3 bits** in the MSB indicate the pages number, whereas the remaining **13 bits** indicate the offset within that page:

Program view of the virtual address space



Virtual address
0x325A

0011 0010 0101 1010

0011 0010 0101 1010

page number

displacement

OS view of the virtual address space

page 0	0x0000
page 1	0x2000
page 2	0x4000
page 3	0x6000
page 4	0x8000
page 5	0xA000
page 6	0xC000
page 7	0xE000
	0xFFFF

Virtual address space and Memory address space

The page number is then **1** and the offset is **0x135A**:

Program view of the virtual address space



Virtual address
0x325A

0011 0010 0101 1010

0011 0010 0101 1010

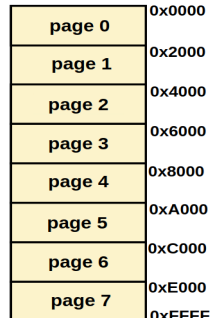
page number

1

displacement

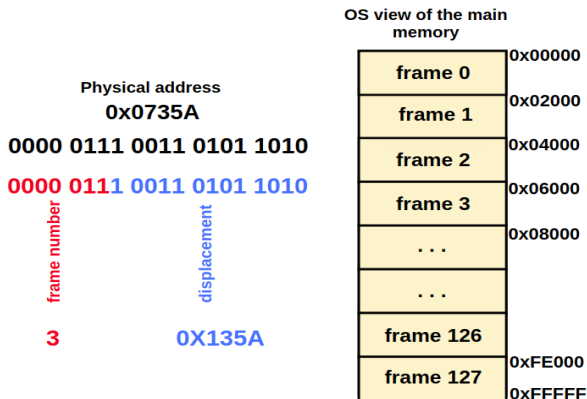
0X135A

OS view of the virtual address space



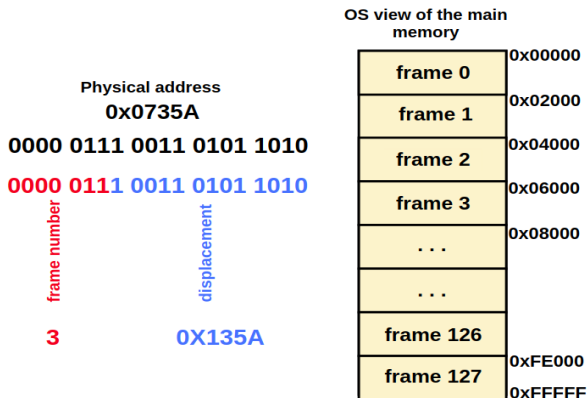
Virtual address space and Memory address space

Physical addresses are on **20 bits** and the physical address space is divided into frames of **8KB** each: **We have a total of 128 frames.**



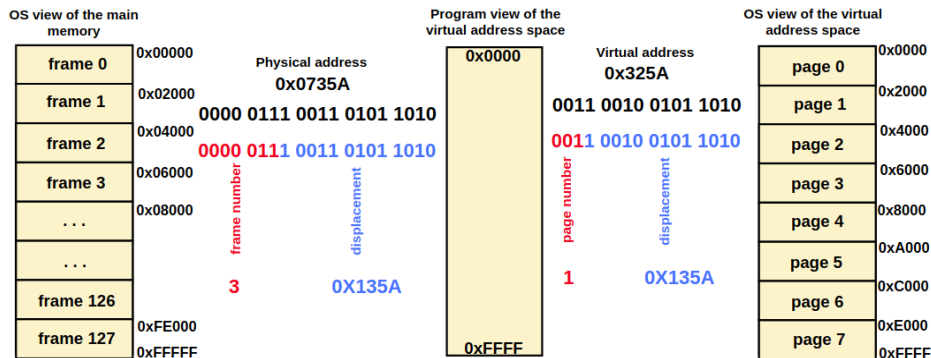
Virtual address space and Memory address space

We need **7 bits** for frame number and **13 bits** for offset.



Virtual address space and Memory address space

Address translation converts **16 bits** virtual address to a **20 bits** physical address.



Actually, address translation converts a **3 bits** page number into a **7 bits** frame number.

Memory protection in paged environment

Paging uses a particular bit known as **valid-invalid bit** for each entry in the page table.

A **valid-invalid bit** in a given page table entry indicates whether the associated page is in the process's logical address space or not.

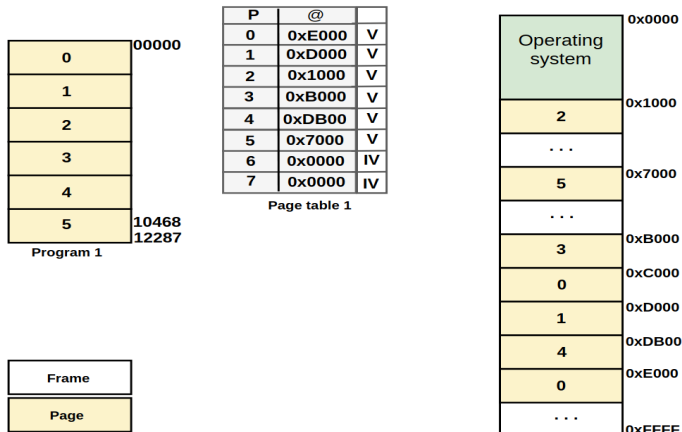
I.e., if a process tries to access a page that is not valid, a trap is raised.

The operating system sets this bit for each page entry to allow or disallow access to a given page.

Also, each page table entry contains other property bits such as a present bit, a dirty or modified bit, or process ID information, depending on OS.

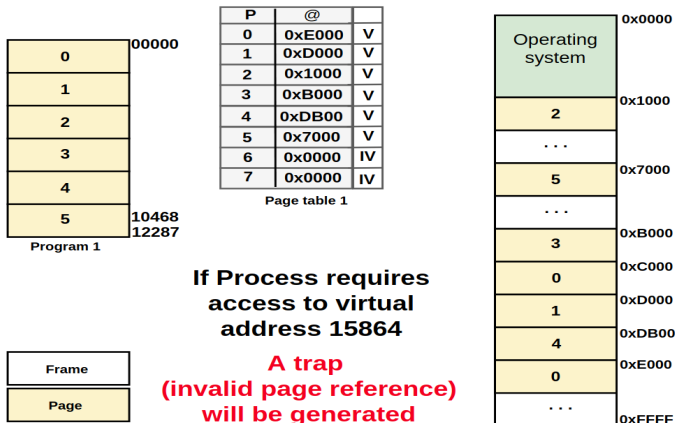
Memory protection in paged environment

Example. Consider a 14-bit computer and a program that uses only addresses from **0** to **10468** (i.e., virtual address space in decimal). And consider that the page size is set to **2KB (8 pages)**.



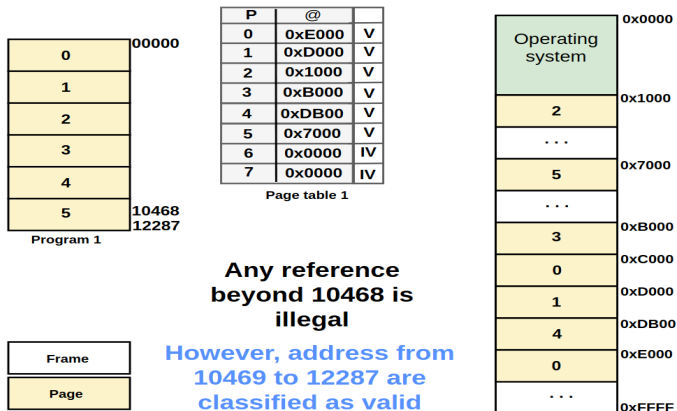
Memory protection in paged environment

Example. Consider a 14-bit computer and a program that uses only addresses from **0** to **10468** (i.e., virtual address space in decimal). And consider that the page size is set to **2KB (8 pages)**.



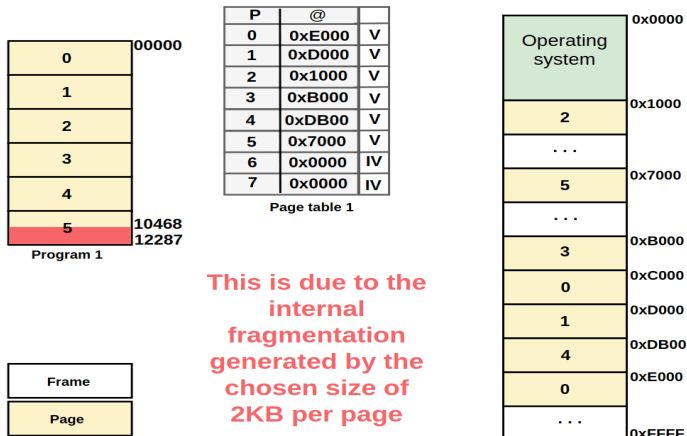
Memory protection in paged environment

Example. Consider a 14-bit computer and a program that uses only addresses from **0** to **10468** (i.e., virtual address space in decimal). And consider that the page size is set to **2KB (8 pages)**.



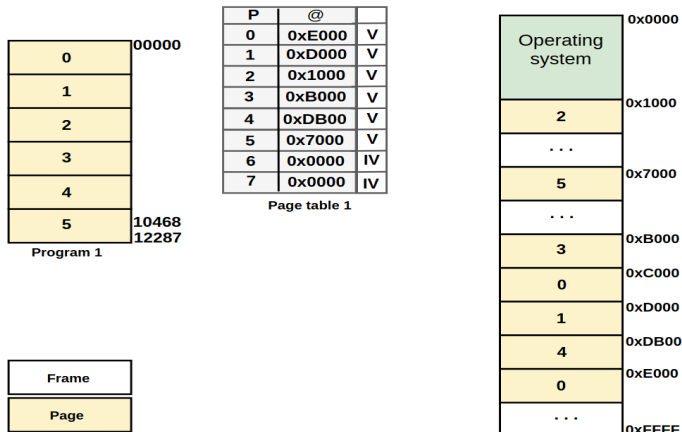
Memory protection in paged environment

Example. Consider a 14-bit computer and a program that uses only addresses from **0** to **10468** (i.e., virtual address space in decimal). And consider that the page size is set to **2KB (8 pages)**.



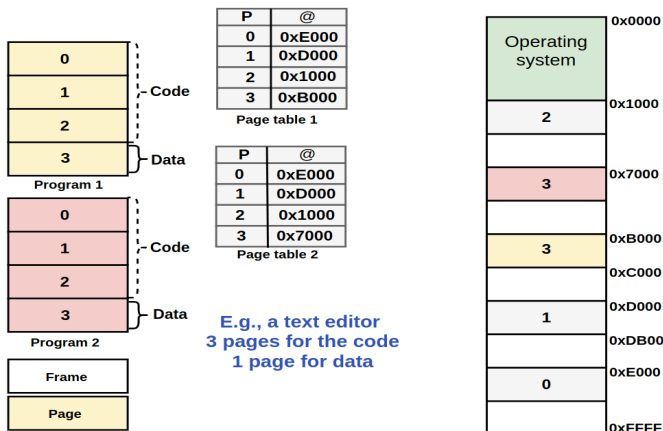
Memory protection in paged environment

Rarely does a process use all its address range. It would be wasteful in this case to create a page table with entries for every page in the virtual address range. Some systems use a PTLR (Page Table Length Register).



Page sharing in paged environment

One of the advantages of paging is the possibility of sharing pages among processes (**virtual addresses mapped to the same physical addresses**).



Memory Management

Memory Management: Multilevel Paging

Single-level Paging

Single-level paging (paging), is the non-contiguous memory management scheme that consists of:

- Dividing the virtual memory space of each program into pages and dividing the central physical memory into frames to host programs' pages.
- Each program would have a page table that tells where each of the program's pages is stored in which frame in the central physical memory.
- A program may not use all the possible virtual address space (i.e., all possible pages). This results in invalid entries in the page table.
- If a page is not loaded in a frame, and the program requests access to it, then a page fault exception is raised, and the missing page is loaded.
- It allows large programs (e.g., 4GB) to run in small RAMs (e.g., 512 MB).

However, on a 32-bit operating system that uses 4KB page size, and where a page entry stores 16 bytes of information, each program would have a **flat** page table of size: 16 MB, even though the size of the program is less than 16 MB.

Multilevel Paging

Modern computers support large logical address space (e.g., 2^{32} to 2^{64}) and this may result in large page tables stored in the memory.

i.e., to access one byte of the process memory, we need to have the whole page table stored in the RAM. This is not efficient in modern computers.

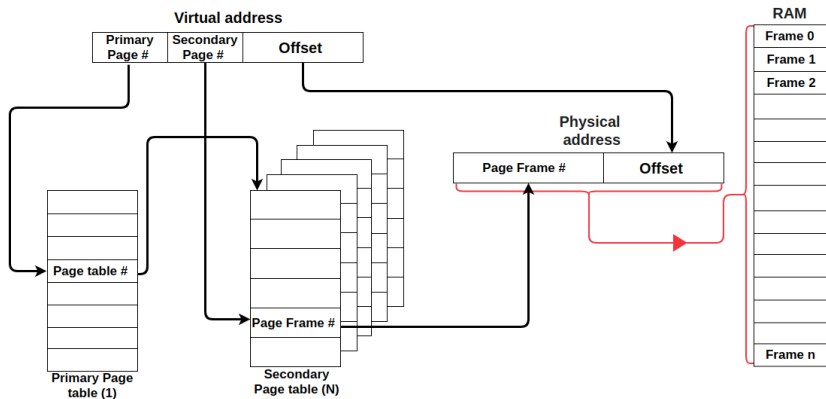
One approach to solve this issue is to load into the memory a portion of the page table and load the other portion(s) when needed.

This is called, multilevel paging. It consists of structuring the page table into a tree of page tables (i.e., multiple levels).

- The DEC PDP 11 uses 1-level paging (single-level paging).
- The DEC VAX computer uses 2-level paging scheme.
- The SUN SPARC computer uses 3-level paging scheme.
- The Motorola 68030 computer uses 4-level paging scheme.

Multilevel Paging (2-level paging)

In this scheme, the page number part is split into two equal sized parts. The first part encodes an entry in the primary page table (Level 1), whereas the second part encodes an entry in the secondary page table (Level 2).



Multilevel Paging (2-level paging)

Example. Consider a page size of 4KB in a 32-bit operating system where each PTE (Page Table Entry) is 4 bytes and single-paging scheme is used. Then, **What is the size of a page table?**

- Each process sees 2^{32} addresses, i.e., 4GB of virtual address space.
- The total amount of virtual space is divided into 1M pages.
- A virtual address is composed of 20 bits Page# and 12 bits Offset.
- The page table will be composed of 1M entries.
- The size of a page table (flat page table) is:

$$1\text{M} \times 4\text{B} = 4\text{MB}$$

Which means, each process will need 4MB to store its page table. This also means, we need 4MB to access any data of the process.

Single-paging vs 2-level paging

A virtual page will have the form [page#1, page#2, Offset]. For example, in a 32-bit operating system that uses 4KB of page size, the virtual addresses will be interpreted as:

$$\underbrace{(b_0, b_1, \dots, b_9)}_{\text{page number 1}}, \underbrace{(b_{10}, b_{11}, \dots, b_{19})}_{\text{page number 2}}, \underbrace{(b_{20}, b_{21}, \dots, b_{31})}_{\text{displacement}})_2$$

Thus, for a given virtual address, the CPU uses the Level 1 page table along with the 1st page number (i.e., $\underbrace{(b_0, b_1, \dots, b_9)}_{\text{page number 1}})_2$) to find the physical address

of the Level-2 page table.

It then uses the 2^{ed} page number (i.e., $\underbrace{(b_{10}, b_{11}, \dots, b_{19})}_{\text{page number 2}})_2$) to determine the

corresponding frame number (i.e., the base address of the frame). The final physical address is:

$$(\underbrace{b_0, b_1, \dots, b_{N-1}}_{\text{page number 1}})_2 + (\underbrace{b_{20}, b_{21}, \dots, b_{31}}_{\text{displacement}})_2$$

Single-paging vs 2-level paging

Example. Consider a page size of 4KB in a 32-bit operating system where each PTE (Page Table Entry) is 4 bytes and a two level-paging scheme is used. Then, **what is the size of a page table?**

- Each process sees 2^{32} addresses, i.e., 4GB of virtual address space.
- The total amount of virtual space is divided into 1M pages.
- A virtual address is composed of 10 bits Page# 1, 10 bits Page# 2, and 12 bits Offset.
- Each page table will be composed of 1K entries.
- The size of one page table at a given level is:

$$1K \times 4B = 4KB$$

i.e., each process will need 8KB of memory space to access any data of the process, 4KB of the 1st level page, and 4KB of the 2nd level page.

Two-level Paging

Exercise. Consider a page size of 1MB in a 32-bit operating system where each PTE (Page Table Entry) is 16 bytes and a two level-paging scheme is used. Then, **what is the size of all page tables of a 512MB program?**

Possible answers:

9KB

32KB

64KB

65KB

512KB

1MB

Two-level Paging

Exercise. Consider a page size of 1MB in a 32-bit operating system where each PTE (Page Table Entry) is 16 bytes and a two level-paging scheme is used. Then, **what is the size of all page tables of a 512MB program?**

Possible answers:

9KB ✓

32KB

64KB

65KB

512KB

1MB

- End.